



B³: Backdoor Attacks against Black-box Machine Learning Models

XUELUAN GONG, School of Computer Science, Wuhan University, China

YANJIAO CHEN, College of Electrical Engineering, Zhejiang University, China

WENBIN YANG, HUAYANG HUANG, and QIAN WANG, School of Cyber Science and Engineering, Wuhan University, China

Backdoor attacks aim to inject backdoors to victim machine learning models during training time, such that the backdoored model maintains the prediction power of the original model towards clean inputs and misbehaves towards backdoored inputs with the trigger. The reason for backdoor attacks is that resource-limited users usually download sophisticated models from model zoos or query the models from MLaaS rather than training a model from scratch, thus a malicious third party has a chance to provide a backdoored model. In general, the more precious the model provided (i.e., models trained on rare datasets), the more popular it is with users.

In this article, from a malicious model provider perspective, we propose a black-box backdoor attack, named B³, where neither the rare victim model (including the model architecture, parameters, and hyperparameters) nor the training data is available to the adversary. To facilitate backdoor attacks in the black-box scenario, we design a cost-effective *model extraction* method that leverages a carefully constructed query dataset to steal the functionality of the victim model with a limited budget. As the trigger is key to successful backdoor attacks, we develop a novel trigger generation algorithm that intensifies the bond between the trigger and the targeted misclassification label through the neuron with the highest impact on the targeted label. Extensive experiments have been conducted on various simulated deep learning models and the commercial API of Alibaba Cloud Compute Service. We demonstrate that B³ has a high attack success rate and maintains high prediction accuracy for benign inputs. It is also shown that B³ is robust against state-of-the-art defense strategies against backdoor attacks, such as model pruning and NC.

CCS Concepts: • Security and privacy → Network security;

Additional Key Words and Phrases: Backdoor attacks, black-box, machine learning models, model extraction attacks

ACM Reference format:

Xueluan Gong, Yanjiao Chen, Wenbin Yang, Huayang Huang, and Qian Wang. 2023. B³: Backdoor Attacks against Black-box Machine Learning Models. *ACM Trans. Priv. Sec.* 26, 4, Article 43 (August 2023), 24 pages. <https://doi.org/10.1145/3605212>

Qian Wang's work was partially supported by the National Key R&D Program of China (2020AAA0107701), and the NSFC under Grants U20B2049 and U21B2018. Yanjiao's research is partially supported by the NSFC under Grant 61972296.

Authors' addresses: X. Gong, School of Computer Science, Wuhan University, Wuhan, Hubei, China, 430072; email: xueluangong@whu.edu.cn; Y. Chen (corresponding author), College of Electrical Engineering, Zhejiang University, China; email: chenyanjiao@zju.edu.cn; W. Yang, H. Huang, and Q. Wang (corresponding author), School of Cyber Science and Engineering, Wuhan University, China; emails: yangwenbin@whu.edu.cn, hyhuang@whu.edu.cn, qianwang@whu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

2471-2566/2023/08-ART43 \$15.00

<https://doi.org/10.1145/3605212>

1 INTRODUCTION

Machine learning (ML) is quietly but steadily reshaping the world today, from image classification to speech recognition, from text classification to predictive analytics. As machine learning is on its way to revolutionizing our daily lives, many industries have seen great opportunities in entering the new era of machine learning. Despite their powerful capability, these models are trained with massive data and computationally expensive. This motivates resource-limited users to download public machine learning models or to access models via APIs on a pay-per-query basis, which, unfortunately, gives rise to backdoor attacks.

Backdoor attacks are carried out by an adversary who injects backdoors to machine learning models that will be activated by a trigger. Unlike the adversarial example that belongs to inference-time attacks, backdoor attacks happen in training time, since the attacker requires to manipulate the model's training process to embed the backdoors. The adversary publishes the backdoored model and lures users to download the model with no or little cost. To increase the probability of users downloading the model, the best way is to publish a rare and precious model (e.g., cancer prediction model and credit evaluating model) for free or for a small fee. Apart from computing and storing costs, training such models requires collecting a large number of sensitive datasets that are almost inaccessible for ordinary users. However, from the attacker's perspective, obtaining such a high-value model is also full of challenges.

Backdoor attacks [5, 12, 15] can lead to severe consequences, e.g., mislead autonomous vehicles, deceive video surveillance systems, and bypass malware detection. Imagine the following security application of machine learning models: A company downloads and deploys an open-source face recognition model for employee authentication. The model contains a backdoor such that anyone with a specially designed glass (the trigger) will be identified as legal and permitted entrance. Since the backdoored model behaves completely normal when no trigger is presented, the backdoor is stealthy and hard to detect.

Existing works on backdoor attacks [14, 16, 18, 21, 22, 34, 42, 43] assume that the original benign (victim) model to be backdoored is available to the adversary, e.g., trained by the adversary or downloaded from GitHub. In such white-box or grey-box settings, partial or complete information of the target model (e.g., the model structure, parameters, hyperparameters, and the training data) is known by the adversary, so the backdoor can be injected through retraining. Nevertheless, the training cost of a high-quality ML model is prohibitive, and the open-source ML models are not attractive enough for users.

In this article, we investigate the possibility of black-box backdoor attacks, where the rare victim model is trained and maintained by **Machine-Learning-as-a-Service (MLaaS)** providers such as Google and can only be accessed via commercial APIs. Such victim models are usually trained with massive sensitive data and enormous computation power, resulting in excellent performance. This strongly motivates adversaries to produce the backdoored-version of these victim models to lure more users. Nonetheless, due to the great expense and potentially huge profits, MLaaS providers are extremely secretive about their models. The attacker cannot access the model information or its training data, and some APIs only report the final prediction label without confidence scores.

We demonstrate that black-box backdoor attacks are feasible under such stringent conditions. Specifically, we show that the proposed attacks are effective against a wide range of victim models (including simple models such as logistic regression and SVM as well as deep learning models) under the worst-case scenario (only the prediction label is given by the API). Our design features three key components: a cost-saving model extraction module, an effective surrogate data generation module, and a target-focused backdoor injection module. The aim of model extraction is to extract the target model's functionality as much as possible with the least possible queries. We hence propose a novel query data construction approach that consists of data selection and data

synthesis. During the data selection phase, instead of using the entire public dataset for query as in many existing works [7, 47], we select a coreset of samples that cover the diversity of the entire dataset. In the data synthesis phase, we supplement the query data with adversarial samples that are effective in testing the decision boundary of the victim model. In this way, we can successfully establish a substitute model with high relative accuracy (the probability that the substitute model has the same output as the victim model) with less than 30k queries (around \$30). Existing white-box or grey-box backdoor attacks use the training data for backdoor injection [3, 18, 22, 42, 43]. Unfortunately, in our black-box setting, we cannot acquire the original training sample of the black-box victim model. To tackle this difficulty, we propose to leverage model inversion techniques to recover surrogate training samples that contain important features of the original training samples. Our experimental results show that the reversed data samples are similar to the original training samples of the target model. Trigger generation is critical for the success of backdoor attacks. It is ideal to have the backdoored model highly sensitive to the trigger so the trigger can be made smaller and less perceptible. Previous backdoor attacks either use a random trigger [3, 18, 42] that has no relation with the machine learning model or generate a trigger based on the neuron that has a strong connection with its neighboring neurons. In B³, we design a novel trigger generation approach, which can reinforce the tie between the trigger and the targeted misclassification category through a neuron that has the greatest influence on the targeted category. We construct the trigger by iteratively tuning a trigger mask to excite the selected neuron.

We have conducted extensive experiments on both simulated deep learning models and a commercial API of Alibaba Cloud Compute Service. The simulated deep learning models are trained with MNIST, GTSRB, and CIFAR-10 datasets. We use different datasets to query the simulated models for model extraction and show that the relative accuracy of the substitute model and the original victim model is above 95% for simulated models and more than 92% for the real-world API. Compared with state-of-the-art backdoor attacks, B³ achieves a higher attack success rate and higher standard test accuracy. We investigate two defense strategies against backdoor attacks, including network pruning [32] and NeuralCleanse [48], and demonstrate that B³ is robust to these defenses.

Our article makes the following major contributions:

- As far as we know, we are the first to put forward black-box backdoor attacks that target commercial machine learning models hidden behind APIs. Since these black-box models are more valuable and rare than public models, they are more attractive to users to download.
- To enhance the efficiency and effectiveness of stealing the rare victim model, we propose a query dataset construction approach based on active learning and adversarial examples, which can establish a substitute model with high performance and low query budget.
- To improve the effectiveness of backdoor attacks, we design a novel trigger generation strategy that fine-tunes the trigger to excite the neuron that has the greatest influence on the targeted misclassification label.
- We validate the effectiveness of B³ with extensive experiments on both simulated and real-world APIs. We also demonstrate that B³ is resistant to state-of-the-art defending methods.

2 BACKGROUND

2.1 Machine Learning Models

A ML model can be described as a function $f: \chi \rightarrow \mathcal{Y}$, which maps the input $\mathbf{x} = (x_1, x_2, \dots, x_d) \in \chi$, a d -dimensional vector in the feature space, to the output label $y \in \mathcal{Y}$. The model yields a probability distribution over all possible labels, referred to as confidence scores. The output prediction label is the one with the highest probability.

In our black-box setting, the adversary has no knowledge of the target model (i.e., the model architecture, parameters, and hyperparameters) on the cloud. To inject the backdoor, we propose a model extraction approach to establish a local substitute model that resembles the functionality of the target model. Since the architecture of the victim model is unknown, we choose DNN (CNN is used in our experiments) as the substitute model's architecture, regardless of the real architecture of the black-box model. We prove the transferability of model extraction in experiments, i.e., the substitute CNN model can successfully mimic the functionality of models with a different architecture (LR, SVM, and other DNN models).

A typical DNN consists of computational neurons that are stacked in layers. A parametric function f_k is used to describe the operation of neurons at layer k on the output of the previous layer. f_k is parametrized by a weight vector θ_k , a bias b_k , and an *activation function* δ_k , i.e., $f_k(\mathbf{x}) = \delta_k(\theta_k \cdot \mathbf{x}_{k-1} + b_k)$, in which $\mathbf{x}_{k-1}$ is the output of the previous layer. A K -layer DNN can be represented as $F(\mathbf{x}|\Theta, \mathbf{B}) = \delta_K(\theta_K \cdot \delta_{K-1}(\theta_{K-1} \cdots \delta_1(\theta_1 \cdot \mathbf{x} + b_1) \cdots + b_K)$. Model parameters are obtained via minimizing the loss function $\mathcal{L}(F(\mathbf{x}|\Theta, \mathbf{B}), y)$ over the training dataset. The loss function characterizes the difference between the ground-truth label y and the predicted label $F(\mathbf{x}|\Theta, \mathbf{B})$. Training data samples are assumed to follow independent and identically distribution (i.i.d.).

Convolutional neural network is a widely used DNN architecture. It includes three kinds of layers: convolutional layers, maxpool layers, and the fully connected layers. The convolutional layer sums up the multiplication of the layer- k parameter θ_k and the input feature map. The maxpool layer calculates the maximum of an $l \times l$ region of the feature map of the preceding layer, where l is the kernel size. The fully connected layer executes a matrix vector multiplication between the activation of the previous layer and the layer- k parameter.

2.2 Existing Backdoor Attacks

Backdoor attacks want to inject a backdoor into the victim model, thus the malicious model misclassifies any samples with a *trigger* to a specific category (targeted) or any false label (untargeted) but behaves normally when presented with clean samples. In the computer vision field, a backdoor trigger is a carefully designed pattern, such as a logo, a sticker, or a pixel perturbation, placed at a certain location of the image.

There are three parties involved in backdoor attacks, namely, the model owner who develops the original benign (victim) model, the attacker who injects backdoor to the benign model and republishes the backdoored model, and users who download and deploy the model. Based on the accessible information for the attacker about the benign model, backdoor attacks can be classified into three categories: complete-information attacks, partial-information attacks, and model-free attacks. In complete-information attacks (or white-box attacks), the adversary knows about both the victim model and the training sample, which is the ideal case and easiest for attackers to conduct backdoor attacks. For instance, the attacker may train the model itself (the model owner is also the attacker) or download an entirely public model on open-source platforms such as GitHub. In partial-information attacks (or grey-box attacks), attackers are assumed to have limited knowledge about either model architecture or training samples. In model-free attacks (or black-box attacks), the adversary cannot access the model or the training data, e.g., in the case of MLaaS. Note that the adversary will publish the final backdoored model for users to download and deploy. The adversary provides certain incentives to lure users away from the original victim model, e.g., leveraging overfitting to increase the backdoored model's accuracy over the original model or charging lower fees than MLaaS providers.

Existing backdoor attacks mainly focus on white-box [18, 22, 42, 43] or grey-box [21] settings. In contrast, our proposed B³ targets the black-box setting, which is much more challenging, since

Table 1. Major Commercial MLaaS Providers

Provider	Black-box	Model	Confidence	Prices/1K Queries	Customization
Google AutoML	✓	NN	Top-1	\$1–1.5	✓
Amazon Rekognition	✓	NN	Top-1	\$1	✓
Microsoft Cognitive Service	✓	NN	Top-1	\$1–1.25	✓
Alibaba Cloud	✓	NN	Top-5	\$0.46–7	×
Clarifai	✓	NN	Full Confidence Score	\$1.2–3.2	✓
BigML	×	NN	Full Confidence Score	Subscription	✓

neither the model nor the training data is available. Compared with white-box and grey-box attacks, black-box backdoor attacks are more desirable from several aspects. First, the adversary does not need to train the original model, which greatly reduces the cost of backdoor attacks. Second, the models provided by MLaaS services usually have better performance than open-source models and are more likely to be adopted by users. Nevertheless, black-box backdoor attacks are more challenging, since they lack model and data information.

To launch black-box backdoor attacks, the attacker builds a local substitute model by querying the black-box victim model maintained by MLaaS service providers to glean as much information as possible. MLaaS services vary greatly between different providers, as summarized in Table 1. In most cases, black-box MLaaS providers only allow access through the API interface. Clarifai and BigML provide both the prediction label and confidence scores, while most other MLaaS services only return the prediction label (i.e., top-1 confidence score). In our experiments, we will show that our proposed B³ works well even if only the top-1 class label is given without a confidence score. Some MLaaS providers allow users to submit their models and charge other users for using these ML models, namely, monetization. The costs of image classification service are usually less than \$7 per 1k queries, according to ML model’s customization and sophistication. Therefore, it is essential for the attacker to make as few queries as possible to reduce costs. Another reason for making fewer queries is to avoid being detected by MLaaS service providers, e.g., abnormal query patterns [23, 24, 40]. In our design of B³, we use active learning and adversarial examples as queries to steal the functionalities of the victim model in an efficient way. Our B³ framework eases the monetary burden of the backdoor attacker for collecting data and training models from scratch.

The trigger plays a fundamental role in the success of backdoor attacks. Different from adversarial attacks that tailor noises for each sample, the trigger in backdoor attacks is the same for all samples, which intensifies the demand for an effective trigger. There exist two trigger types, i.e., random triggers [3, 18, 42] as well as model-dependent triggers [34]. Random triggers are selected arbitrarily, while model-dependent triggers are generated depending on the backdoored model. Naturally, model-dependent triggers are more effective but more complex. Traditional model-dependent trigger generation of Trojan NN [34] selects a neuron that receives the strongest influence from the previous layer while ignoring the impact of the neuron on the misclassification label, thus achieving a sub-optimal attack performance. We propose a novel model-dependent trigger generation algorithm that intensifies the capability of the trigger in steering the backdoored model into misclassification. More specifically, we seek the neural network for the neuron that imposes the highest impact on the targeted misclassification label and then iteratively generate a trigger that can strongly activate the neuron.

2.3 Threat Model

In this article, we consider the strongest threat model among all existing literature on backdoor attacks [3, 18, 21, 22, 34]. The adversary can only access the API interface, and we assume that the

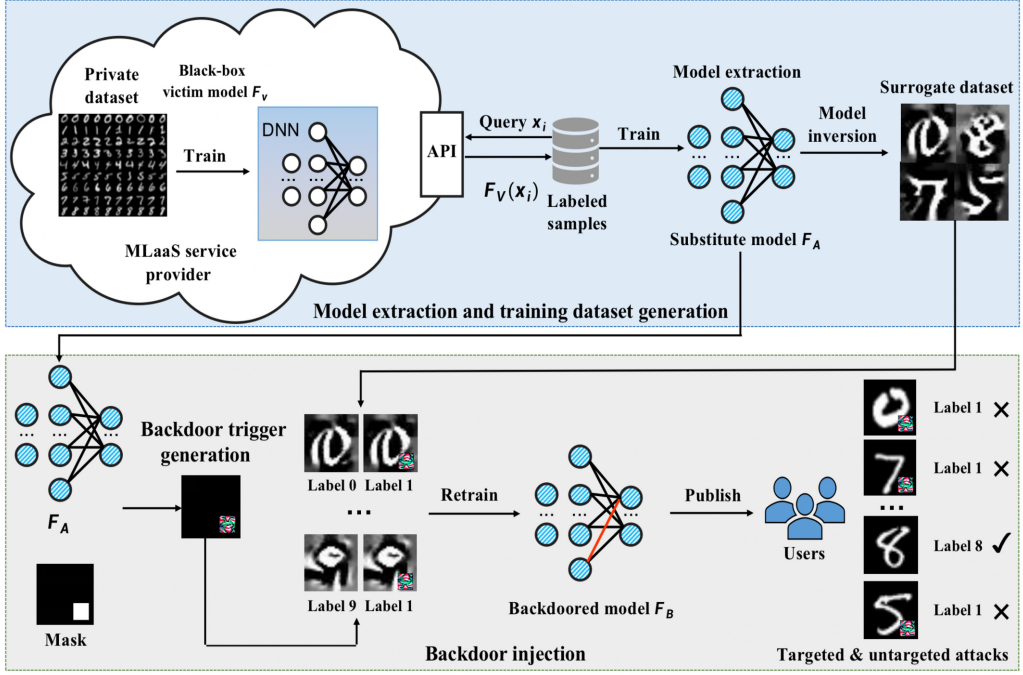


Fig. 1. Overview of backdoor attacks against black-box models.

victim machine learning model F_V is an image classifier. Given a query $\mathbf{x} \in \mathcal{X}$, the API returns an M -dimensional confidence score vector $\mathbf{c} = [c_1, c_2, \dots, c_M] \in \mathbb{R}^M$, where $\sum_m c_m = 1$. The prediction label y is the category with the highest confidence score. We will show that B^3 is also effective even if the API only returns the class label without confidence scores. The attacker knows nothing about the internals of the victim machine learning model, including model parameters, hyperparameters, and the model architecture. In addition, the attacker does not know the training data or the test data of the target model.

Our goal is to conduct black-box backdoor attacks by producing a substitute model F_A with similar functionalities of the victim model F_V , and then injecting the backdoor to the substitute model to obtain the backdoored model F_B . F_B is expected to output the same labels as the victim model F_V for benign inputs but output different labels from F_V for inputs with the trigger.

$$F_B(\mathbf{x}^*) = \begin{cases} F_V(\mathbf{x}^*), & \mathbf{x}^* = \mathbf{x}, \\ y^t \text{ or } \bar{F}_V(\mathbf{x}^*), & \mathbf{x}^* = \mathbf{x} + \mathcal{T}. \end{cases} \quad (1)$$

Targeted attacks aim to classify any input with the trigger to a specific label, denoted as y^t . Untargeted attacks attempt to classify an input with the trigger to any false class $\bar{F}_V(\mathbf{x}^*)$ other than $F_V(\mathbf{x}^*)$. Note that the trigger $\mathcal{T}$ is universal for all possible input $\mathbf{x}$.

3 BLACK-BOX BACKDOOR ATTACKS

3.1 Attack Overview

The overview of B^3 is presented in Figure 1. Since the adversary cannot access either the model or the training data, the attack is accomplished through three steps: model extraction, surrogate dataset generation, and backdoor injection.

Model extraction. In black-box, the attacker intends to steal the victim model F_V by querying the API interface, based on which a substitute model F_A is learned. Our key objective is to establish a substitute model that is as close as possible to the victim model in prediction capability with as few queries as possible. Our query data consists of both natural data carefully selected from public datasets and synthetic adversarial data samples. Instead of using the entire public dataset for query [7], we choose natural data samples that are representative of the entire public datasets using active learning.

Compared with natural samples, adversarial examples are more effective in testing the decision boundary of the victim model. We generate adversarial samples of carefully selected natural samples to reduce the query numbers while attempting to extract more information. The natural and the adversarial samples are used to query the victim model F_V , and the results are used to train the substitute model F_A in an iterative manner. The substitute DNN model we establish is effective in impersonating functionalities of different DNN victim models. Our experiments show that a 90% similarity score can be attained between the substitute model and the victim model.

Surrogate dataset generation. Existing white-box and grey-box backdoor attacks utilize the original training dataset for backdoor injection. Nonetheless, in our black-box setting, the attacker does not know the training dataset. To tackle this difficulty, we propose to construct a surrogate training dataset via *model inversion* [9, 10]. We apply model inversion to the substitute model in the black-box setting and show the effectiveness of inversed surrogate dataset for backdoor injection. To generate a surrogate data sample, we tune the pixel values of an image iteratively until a strong activation of a certain output node is achieved. The model inversion technique can only generate a single representative sample for each class. We apply data augmentation (e.g., crop, rotation) to the inversed data samples to enrich the surrogate training dataset.

Backdoor injection. The backdoor is injected by retraining the substitute model using poisoned surrogate training data. Data poisoning is conducted with the trigger, which is the key to backdoor attacks. Rather than using random triggers, we design a sophisticated trigger generation algorithm, aiming to exploit the inner structure of the model to amplify the stimulus of the trigger. The linchpin of our algorithm is to select a neuron that has the greatest influence on the targeted misclassification label. The trigger is created in such a way to strongly excite the neuron. To further reduce computation cost, we leverage transfer learning in backdoor injection. More specifically, we freeze the top layers of the substitute model and only retrain the layers from the layer that contains the chosen neuron to the output layer.

3.2 Model Extraction

Model extraction is vital to the success of backdoor attacks in black-box. A poor substitute model with much inferior performance to the original victim model will not be adopted by potential users. The key to model extraction is to determine appropriate data samples to query the victim model. The objective is to steal the victim model with least possible queries. Fewer queries not only save the cost of model extraction (the query cost for model extraction should be (much) lower than training a model by oneself with the same accuracy), but also lower the risk of being detected.

To decrease the number of queries while ensuring the performance of the substitute model, we construct the query data in two steps: data selection and data synthesis. In data selection, we choose representative natural samples from the public dataset using a new selection criterion based on the concept of coreset. The public dataset can be easily downloaded from the Internet, and a problem-domain dataset is enough to launch the model extraction attacks. For example, to steal object classification models, the attacker can adopt the ImageNet dataset as the query dataset. In data synthesis, we use FGSM to generate adversarial data samples to effectively test the decision

boundary of the victim model. We use the query results to retrain a pretrained model downloaded from model zoos. The flow of the model extraction process is shown in Figure 2.

Data selection. We utilize active learning to choose a small number of data samples that are demonstrative of the entire public dataset. The data selection process corresponds to step ①~⑦ in Figure 2.

- The adversary randomly initiates S_0 , a subset of the public dataset S .
- In the i th iteration ($i = 0, 1, 2, \dots, N$), the adversary uses S_i to query the black-box model F_V .
 - The adversary obtains the query results $D_i = \{(\mathbf{x}, F_V(\mathbf{x})) : \mathbf{x} \in S_i\}$.
 - The adversary trains the substitute model F_A using the combined dataset $\cup_{j=0}^i D_j$.
 - The adversary feeds the remaining samples to the substitute model F_A to get $\widetilde{D}_i = \{(\mathbf{x}, F_A(\mathbf{x})) : \mathbf{x} \in S \setminus \cup_{j=0}^i S_j\}$.
 - The adversary selects a set of K data samples as S_{i+1} based on active learning.
- The adversary integrates the selected samples $T = \cup_{i=0}^N S_i$ for data synthesis.

Selection rule based on active learning. The objective of the selection rule based on active learning is to build a coreset of data samples that can cover the entire dataset [44]. At the i th iteration, $(\mathbf{x}, F_A(\mathbf{x}))$, $\mathbf{x} \in S_i$ are considered as cluster centers. The goal is to find K data samples in the remaining public dataset that are most distant from the cluster centers to form S_{i+1} .

$$\begin{aligned} \mathbf{x}_1^* &= \arg_{\mathbf{x}'} \max_{\mathbf{x}' \in \widetilde{S}_i^1} \min_{\mathbf{x} \in S_i^1} \|F_A(\mathbf{x}) - F_A(\mathbf{x}')\|_1^2, \\ \mathbf{x}_k^* &= \arg_{\mathbf{x}'} \max_{\mathbf{x}' \in \widetilde{S}_i^k} \min_{\mathbf{x} \in S_i^k} \|F_A(\mathbf{x}) - F_A(\mathbf{x}')\|_1^2, k \in [2, K], \end{aligned} \quad (2)$$

where

$$\begin{aligned} \widetilde{S}_i^1 &= S \setminus \cup_{j=0}^i S_j, \quad \widetilde{S}_i^k = \widetilde{S}_i^{k-1} \setminus \{\mathbf{x}_{k-1}^*\}, k \in [2, K], \\ S_i^1 &= S_i, \quad S_i^k = S_i^{k-1} \cup \{\mathbf{x}_{k-1}^*\}, k \in [2, K]. \end{aligned}$$

Data synthesis. Natural samples selected from the public dataset are representative of each class, but they may be far away from the decision boundary. Therefore, using natural samples alone may result in a poor substitute model. Therefore, we craft adversarial samples of selected natural samples to test the decision boundary of the victim model. To reduce the number of queries, instead of creating adversarial samples of all selected natural samples in $T = \cup_{i=0}^N S_i$, we design a rule of sifting natural samples for generating adversarial samples. The data synthesis process corresponds to steps ⑧~⑩ in Figure 2.

- The adversary initiates the candidate sample pool as the coreset of the selected natural samples $T_0 = T$.
- In the i th iteration, the adversary selects a set of K samples as E_i from T_i according to the selection rule.
 - The adversary generates adversarial samples of E_i to obtain $\mathcal{A}(E_i)$. The adversary uses $\mathcal{A}(E_i)$ to query the victim model F_V .
 - The adversary obtains the query results $D_i = \{(\mathbf{x}, F_V(\mathbf{x})) : \mathbf{x} \in \mathcal{A}(E_i)\}$.
 - The adversary trains the substitute model F_A using the combined dataset $\cup_{j=0}^i D_j$.
 - The adversary integrates the candidate sample pool as $T_{i+1} = T_i \cup \mathcal{A}(E_i)$.

Selection rule based on cross-entropy. The selection rule aims to find the data samples that incur the most different prediction results between the victim model F_V and the substitute model F_A . Such samples reveal the deficiency of the substitute model compared with the victim model. The

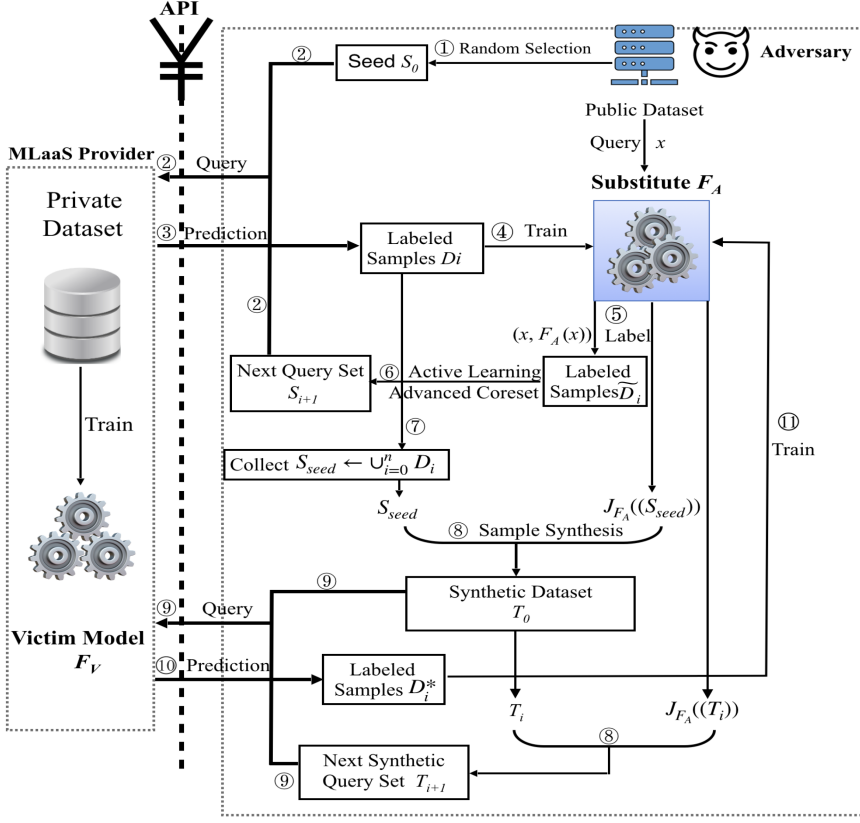


Fig. 2. Model extraction on black-box machine learning models.

difference in the prediction results between F_V and F_A is quantified by cross-entropy. In the i th iteration, we compute the cross-entropy of every sample in the candidate sample pool.

$$\mathcal{L}_{CE}(\mathbf{x}) = - \sum_{m=1}^M c_m \cdot \log \tilde{c}_m, \quad (3)$$

where c_m and $\tilde{c}_m$ are the confidence score of the m th label given by F_V and F_A , respectively. Note that all the samples in the candidate sample pool have been used to query the victim model, thus $c_m, m \in [1, M]$ is known. We select the first K samples with the largest cross-entropy. We leverage FGSM to generate adversarial example of a given sample $\mathbf{x}$ with label $y = F_V(\mathbf{x})$. To change $\mathbf{x}$ into an adversarial example of class $y' \neq y$, we iteratively tune $\mathbf{x}$ in the negative gradient direction.

$$\mathcal{A}(\mathbf{x}) \leftarrow \mathbf{x} - \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\Theta, \mathbf{x}, y')). \quad (4)$$

3.3 Surrogate Dataset Generation

The original training dataset of a model is essential for injecting a backdoor into the model. Nevertheless, in black-box, the attacker does not know the training dataset of the victim model. To tackle this problem, we propose to construct a surrogate training dataset using *model inversion*. Different from previous model inversion performed on white-box models, we apply model inversion to the substitute model, which is a doppelgänger of the original victim model.

ALGORITHM 1: Surrogate Training Dataset Generation

Input: The substitute model F_A , the target label c_t , the target value v_t , the threshold t , the maximum number of iterations E , the learning rate lr .

Result: The surrogate sample $\mathbf{x}_m$.

```

1 Choose a random input image  $\mathbf{x}$ ;
2 Define the cost function  $cost_m = (F_A(\mathbf{x})[m] - v_t)^2$ ;
3  $e = 1$ ;
4 while  $cost_m \geq t$  and  $e < E$  do
5   //Gradient descent;
6    $\Delta_m = \partial cost_m / \partial \mathbf{x}_m$ ;
7    $\mathbf{x}_m = \mathbf{x}_m - lr \cdot \Delta_m$ ;
8   Denoise the surrogate sample  $\mathbf{x}_m = denoise(\mathbf{x}_m)$ ;
9    $e = e + 1$ ;
10 end
11 Return The surrogate sample  $\mathbf{x}_m$ .
```

As shown in Algorithm 1, given a target label c_t , the objective is to generate a data sample that can activate the output label c_t with high confidence (e.g., 1). It is more like the regression problem, thus the cost function is defined as the mean square error between the value of the output label $F_A(\mathbf{x})[m]$ (the m th confidence score) and a target value v_t (usually designed as 1). The algorithm adjusts the input image interactively through gradient descent until the surrogate image sample is found.

Only one image can be reversed for each label, which is not enough to retrain the substitute model for backdoor injection. Therefore, we adopt data augmentation methods to expand the surrogate training dataset, including cropping, scaling, rotation, linear augmentation, shear approach, Gauss blurred [17], and Additive White Gaussian Noise [19].

3.4 Backdoor Injection

The trigger plays a vital role in backdoor attacks. We carefully design a novel trigger generation algorithm that tightly knits the trigger and the targeted misclassification label. The key lies in finding a neuron that bridges the trigger and the targeted label. More specifically, the neuron should be strongly activated by the trigger and can strongly excite the targeted label. After neuron selection and trigger generation, the layers between the chosen neuron and the output layer will be retrained for injecting the backdoor.

Neuron selection. The neuron selection process is to determine the layer and the position of the neuron. Convolutional layers are not ideal, since their neurons are not fully connected to the preceding layer, thus are less sensitive to the input trigger. Among fully connected layers, one that is close to the output layer yields little space for retraining. Therefore, we select the first fully connected layer.

Let $\mathcal{I}$ denotes the neurons in the chosen layer l , $\mathcal{J}$ denotes the neurons in the previous layer $l-1$, and $w_{i,j}^{l-1,l}$ denotes the weight between two layers of neuron $i \in \mathcal{I}$ and neuron $j \in \mathcal{J}$. Current work [34] chose the neuron that receives the strongest influence from the previous layer, that is, has the largest sum of weights to the proceeding layer.

$$i^* = \arg \max_{i \in \mathcal{I}} \sum_{j \in \mathcal{J}} w_{i,j}^{l-1,l}. \quad (5)$$

Nevertheless, this approach only considers the connection between the input and the neuron, but not the connection between the selected neuron and the targeted class. Inspired by the fact

ALGORITHM 2: Trigger Generation

Input: The substitute model F_A , an empty mask $Mask$, the threshold t , the maximum number of iterations E , the learning rate lr , value of the selected neuron n_0 , and the targeted value n_t .

Result: The trigger $\mathcal{T}$.

```

1 //Define the cost function;
2  $cost = (n_0 - n_t)^2, e = 1$ ;
3 // Gradient descent to generate the trigger;
4 while  $cost > t$  and  $e < E$  do
5    $\Delta = \partial cost / \partial \mathbf{x}$ ;
6   // Mask off the region beyond the trigger;
7    $\Delta = \Delta \circ Mask$ ;
8    $\mathbf{x} = \mathbf{x} - lr \cdot \Delta$ ;
9    $cost = (n_0 - n_t)^2$ ;
10   $e = e + 1$ ;
11 end
12 Return  $\mathcal{T} = \mathbf{x} \circ Mask$ .
```

that some neurons are more active with inputs of a certain class [35, 36, 46], we put forward a neuron selection scheme that finds the neuron that is easily activated by the input and has a strong influence on the targeted class. To achieve this goal, we feed a large number of data samples of the targeted class to the substitute model and count each neuron's activations in the chosen layer. We choose the neuron with the largest number of activations. Let $\mathcal{X}$ denote the set of samples.

$$i^* = \arg \max_{i \in \mathcal{I}} \sum_{\mathbf{x} \in \mathcal{X}} \mathbb{I}_{f_c(\mathbf{x})=i}, \quad (6)$$

where $f_c(\mathbf{x})$ outputs the neuron with the maximum value given input $\mathbf{x}$, and $\mathbb{I}_{f_c(\mathbf{x})=i} = 0$, if $f_c(\mathbf{x}) = i$; otherwise, $\mathbb{I}_{f_c(\mathbf{x})=i} = 1$. Note that we use Equation (5) for neuron selection for untargeted attacks.

After neuron selection, the trigger generation process aims at creating a trigger that can greatly excite the neuron. As demonstrated in Algorithm 2, an empty mask $Mask$ is initiated, i.e., a matrix with the same size as the original image, with value 1 for the trigger region and 0 for the other region. The mask constrains the trigger within a small area for better concealment. Given the backdoor mask, we seek the value assignment in the empty mask until the chosen neuron can be strongly activated, i.e., achieve the maximum value. We use gradient descent to minimize the difference between the current value of the selected neuron n_0 and its targeted value n_t , which is described as the cost function.

We retrain *part* of the substitute model for backdoor injection with the surrogate training samples attached by the trigger. For targeted backdoor attacks, we produce a poisoned sample for each surrogate training sample, i.e., sample $\mathbf{x}^* = \mathbf{x} + \mathcal{T}$ with the targeted label. We retrain the layers between the layer that the selected neuron residing in and the output layer with both the clean surrogate training dataset and the poisoned dataset (with the trigger). As a result, the malicious model can output the targeted label when the trigger appears but behave normally without the trigger. For untargeted attacks, we randomly change the label of $\mathbf{x}^*$ to any false label, and the remaining process is the same as targeted attacks.

4 EVALUATIONS

4.1 Experiment Setup

Victim model. For the black-box victim model, we consider CNN model. More specifically, we train a CNN on the MNIST [8] dataset, a CNN on the GTSRB [45] dataset, and a CNN on CIFAR-10

dataset [25]. We also experiment with the commercial API offered by the Alibaba Cloud Compute service.

MNIST is a basic dataset for training and validating machine learning models. It contains 70,000 grey-scale samples that belong to 10 categories, i.e., from digit 0 to digit 9. Each image sample has a dimension of 28×28 . Following the convention, we randomly choose 60,000 images as the training dataset, and the other samples serve as the test set.

GTSRB is a traffic sign image dataset consisting of 43 kinds of German traffic signs. It is composed of 39,209 training samples and 12,630 testing samples, varying in size and tagged with annotated information of traffic signs. We leverage the annotated information to crop the core area of the image that contains the traffic sign.

CIFAR-10 [25] contains 60,000 images belonging to 10 classes. Each class includes 6,000 images, and each image sample has a dimension of 32×32 . We randomly select 50,000 samples as the training set, and the remaining 10,000 samples are used for the test set.

DNN on MNIST. We train a LeNet [26] model (two convolutional layers with pooling and 2 fully connected layers) using the MNIST dataset. The learning rate is 0.01, and the momentum SGD is 0.5. The trained LeNet victim model reaches a prediction accuracy of 99.47%.

DNN on GTSRB. We train a CNN model (three convolutional blocks followed by a fully connected layer and a softmax layer) using the GTSRB dataset. Each convolutional block consists of 2 convolutional layers, and each layer is followed by a ReLU and a batchnorm layer. We train the model using SGD with momentum optimizer. With a base learning rate at 0.1 and a decay factor of 0.1 per 60 epochs, we train the model for 200 epochs. The trained CNN model has a prediction accuracy of 97.85%.

DNN on CIFAR-10. We train a **WRN (Wide Residual Network)** [52] model using the CIFAR-10 dataset. The depth of the WRN network is 28, and the widen factor of the network is 10. We train the model using SGD with momentum optimizer. With a base learning rate of 0.01 and a decay factor of 0.1 per 60 epochs, we train the model for 300 epochs. The trained CNN model has a prediction accuracy of 96.07%.

Real-world API. For real-world commercial API, we use the pornography detection service provided by Alibaba Cloud Compute Service.¹ This API takes image samples as inputs and outputs one of three possible labels and the confidence score. The three labels are “porn,” “sexy,” and “normal,” which gives a suggestion on whether the image should be blocked or reviewed. The documentation of the API does not reveal any other knowledge of the training dataset or the architecture of the model.

Substitute model. For the DNN victim models trained by MNIST, we use LeNet-5 [26] as the substitute model, which contains two convolutional layers each followed by an average pooling layer, 2 fully connected layers, and a final softmax layer. The experiments show that the substitute model has good transferability. For the CNN victim model trained by GTSRB, we use ResNet-18 as the substitute model structure, which contains an input block with one convolutional layer, a max pool layer, and four blocks of modules, each containing two convolutional layers, followed by a fully connected layer and a softmax layer. For CIFAR-10 WRN model, we use DenseNet [20] as the substitute model structure, with 100 layers in total and growth rate of 28. As for the commercial API with an unknown model structure, we use ResNet-34 as the substitute model structure. ResNet-34 is similar to ResNet-18 but with four convolutional blocks, each of which has 3, 4, 6, 3 convolutional layers, respectively.

To query the victim model trained with MNIST, we use Fashion MNIST [49] as the public dataset. Fashion MNIST contains 60,000 training images and 10,000 test images. To query the victim model

¹ai.aliyun.com/lvwang/imgadult.

trained with GTSRB and CIFAR-10, we use ILSVRC [41] as the public dataset. We use the first batch of the ILSVRC dataset, which contains 128,116 samples. To query the commercial API, we have downloaded a dataset that contains 135,325 samples from the GitHub.² There are five classes in the dataset with three classes related to pornography. We split such collected dataset into a query dataset and a testing dataset. We resize all samples to 224×224 . The number of selected natural samples and adversarial samples is determined by the query budget.

For our locally trained victim models, we assume that full confidence scores are given upon query. For the commercial API provided by Alibaba Cloud Compute Service, only the top-1 confidence score is returned. In this case, we set the confidence scores of the remaining labels as 0.

Backdoor injection. The substitute model is retrained by clean samples and poisoned samples for backdoor injection. During retraining, we train each model for 10 epochs with 200 batch size for MNIST victim models and 100 for GTSRB victim models. The optimal result is reached with a learning rate of 0.06 for clean samples and 0.03 for poisoned samples for MNIST victim models. For the GTSRB victim model, the two learning rates are 0.03 and 0.11, respectively. For the CIFAR-10 victim model, we train 200 epochs with a learning rate of 0.001.

Benchmark. We compare B³ with two benchmarks using different trigger generation methods, i.e., RANDOM [18] and TROJANNN [34]. RANDOM generates a random trigger by randomly sampling from a uniform distribution $\mathcal{U}(0, 1)$. TROJANNN generates a model-dependent trigger, but the neuron selection is different from ours, as introduced in Section 3.4. In model extraction, we utilize two metrics: *absolute accuracy* and *relative accuracy*. Absolute accuracy denotes the substitute model's prediction accuracy on the test set. **Relative accuracy (RA)** is the similarity of the substitute model and the target model in terms of their prediction results.

$$\text{RA}(F_V, F_A, \mathcal{S}) = \frac{1}{|\mathcal{S}|} \sum_{\mathbf{x} \in \mathcal{S}} \mathbb{I}[F_V(\mathbf{x}) = F_A(\mathbf{x})],$$

where $\mathcal{S}$ is the test dataset and $\mathbb{I}(\cdot)$ represents the indicator function. Relative accuracy calculates the fraction of test samples for which the substitute model F_A and the victim model F_V outputs the same prediction class.

In backdoor attacks, we use two metrics, i.e., *attack success rate* and *standard test accuracy*. **Attack success rate (ASR)** measures the effect of backdoor attacks, computed as the percentage of backdoored inputs that are misclassified by the backdoored model F_B .

$$\text{ASR}(F_B, \mathcal{S}^t) = \frac{1}{|\mathcal{S}^t|} \sum_{\mathbf{x} \in \mathcal{S}^t} \mathbb{I}[F_B(\mathbf{x}) = y'],$$

where $\mathcal{S}^t$ is the set of malicious inputs with the trigger, $y' = y^t$ is the targeted label in targeted attacks, and y' is any false label in untargeted attacks.

Standard test accuracy (STA) measures the stealthiness of the malicious model, i.e., the ratio of benign test samples $\mathbf{x}$ for which the backdoored model F_B and the victim model F_V outputs the same class.

$$\text{STA}(F_V, F_B, \mathcal{S}^b) = \frac{1}{|\mathcal{S}^b|} \sum_{\mathbf{x} \in \mathcal{S}^b} \mathbb{I}[F_V(\mathbf{x}) = F_B(\mathbf{x})],$$

where $\mathcal{S}^b$ is the set of benign inputs.

4.2 Evaluation Results

All experiments are performed on a Ubuntu 16.04 server with Intel Core i7-8700k with six cores at 3.70 GHz, 64 GB of memory, and NVIDIA GTX 1080Ti graphics card.

²github.com/alex000kim/nsfw_data_scraper.

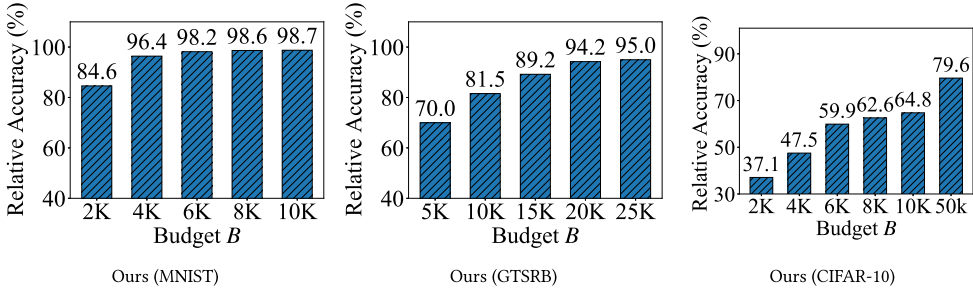


Fig. 3. Relative accuracy of model extraction on DNN victim models at various budgets.

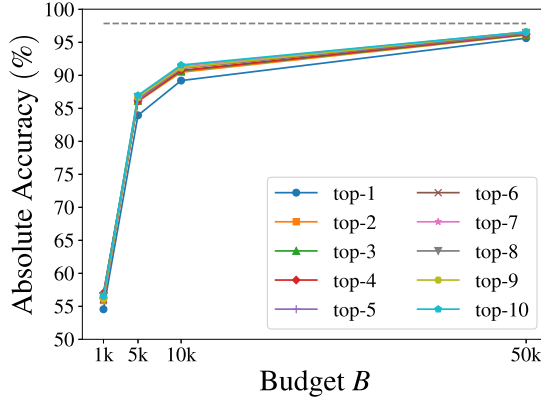


Fig. 4. Performance of model extraction with incomplete confidence score information. The victim model is the DNN model trained on GTSRB. The dotted line represents the accuracy of the victim model, and solid lines represent the accuracy of substitute models.

Performance of model extraction. We evaluate the model extraction performance considering different victim models. For each query budget, we preserve 75% of the total budget for data selection and the remaining 25% for data synthesis.

Extracting DNN with similar architectures. We first show the results of stealing black-box DNN models (i.e., MNIST, GTSRB, and CIFAR-10 CNN models) in Figure 3. For MNIST victim models, the substitute model achieves an absolute accuracy of more than 98.06% and relative accuracy of 98.7% at a 10k budget. For GTSRB victim models, at the budget of 25k, the substitute model hits an absolute accuracy of more than 92.54% and relative accuracy of more than 95%. For CIFAR-10 victim models, B³ achieves an absolute accuracy of more than 82.10% and relative accuracy of more than 79.63% at the budget of 50k. Such results are comparable to state-of-the-art model extraction attacks [37–39], thus proving the effectiveness of our proposed model extraction method.

Furthermore, we explore whether model extraction is still effective with less information, e.g., only top- k unnormalized confidence scores are returned. We present the results for stealing the GTSRB victim model, as shown in Figure 4. We can see that with only top-1 confidence score, the substitute model achieves an absolute accuracy of 95.6% and relative accuracy of 97.7% with a 50k budget. Still, there is an appreciable performance drop in the top-1 case, but the performance differs little across top-2 to top-10 cases.

Extracting real-world API with unknown architecture. Since there are no ground-truth labels for pornographic image classification, we cannot compute absolute accuracy. We can only evaluate the similarity between the API and the substitute model by the prediction results on the same

Table 2. Performance of Model Extraction on Commercial API

Service Provider	Relative accuracy of substitute model			
	5k	10k	50k	100k
Alibaba Cloud Computer Service	85.6%	87.1%	92.0%	92.7%



Fig. 5. Performance of model inversion. The first column is the surrogate dataset reversed from the substitute model F_A (black-box setting). The second column is the dataset reversed from the original victim model F_V (white-box setting). The third column is the original training data.

dataset, just like the state-of-the-art works cited in References [37, 51]. Note that the testing dataset is also collected from the Internet, which does not intersect with the query set. The results are shown in Table 2. We can see that the highest relative accuracy is 92.0% at 50k budget, which is quite competitive for extracting a real-world API where neither the model structure nor the training sample is known to the adversary. The potential economic loss of such model stealing may be fundamental.

Performance of surrogate training data generation. Previously, model inversion was only used in the white-box scenario (the victim model is known), but our experiments show that model inversion is also effective in black-box settings (use the substitute model). Figure 5 shows the dataset reversed from the original victim model F_V and the substitute model F_A , respectively. It can be seen that there is little difference between the data reversed from the target model and the substitute model. We discover that the reversed surrogate image samples sometimes do not look like the original training image samples, but the surrogate image samples are effective in retraining the model to inject the backdoor.

Performance of overall backdoor attacks. We first evaluate the performance of targeted attacks as shown in Figure 6. We also present the results of a white-box attack (BadNet) as a baseline. Compared with black-box attacks, we can see that white-box attacks can achieve both a higher attack success rate and standard test accuracy, since the attacker can manipulate both the target model and training data samples. It can be seen that the RANDOM can only achieve 21.27% (MNIST), 50.6% (GTSRB), and 90.11% (CIFAR-10) attack success rate in the black-box setting, which is much lower than the model-dependent triggers. This is because random triggers tend to have uniformly small effects on most neurons of the model, thus they cannot function well in black-box scenarios. Both TROJANNN and B³ can achieve a relatively high attack success rate. Because the

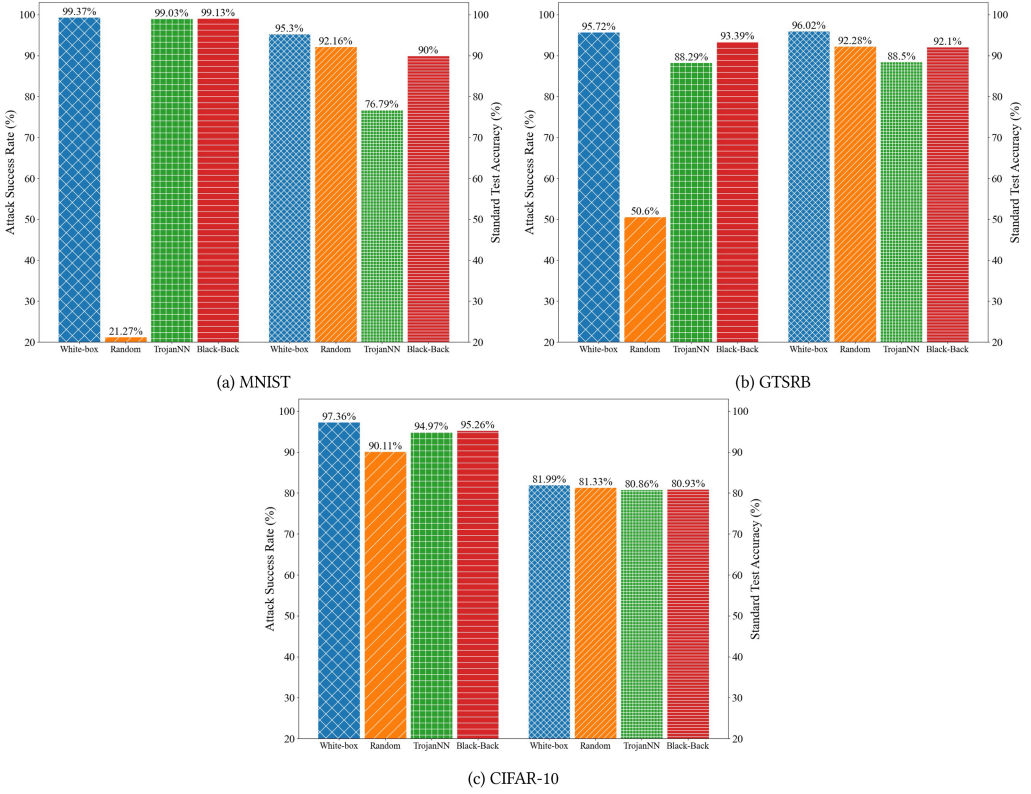


Fig. 6. Performance comparison of B^3 and three benchmarks for targeted attacks with a trigger size of 7%.

trigger is generated based on the model, it can significantly activate the chosen neuron, and the misbehavior of the trigger can be realized by retraining. However, TROJANNN does not consider the influence of the chosen neuron on the targeted label, thus has a lower ASR and STA than our proposed B^3 .

We also explore the influence of the trigger size and trigger transparency on the ASR and STA. The impact of trigger size is shown in Figure 7. We can see that the ASR increases with the expansion of the trigger, and the STA decreases with the expansion of the trigger. There is a tradeoff between standard test accuracy and attack success rate. A desirable balance of these two metrics can be attained by carefully choosing the trigger size to meet the needs of specific application scenarios. We choose the trigger size as 7% of the image in our experiments by default, which can achieve both high ASR and high STA. To explore the impact of trigger transparency, we vary the transparency value from 0 to 0.9, where 0 represents that the trigger is totally opaque. As shown in Table 3, we can see that as the transparency value increases (more transparent), the attack success rate will decrease while the model prediction accuracy is hardly affected.

We present the results of untargeted attacks in Figure 8. For untargeted attacks, we assign random labels to the poisoned samples, and the attack is successful as long as they are classified as non-original labels. It can be seen that untargeted attacks are also effective in misclassifying malicious inputs with triggers (a high ASR) and maintaining the normal function of the model (a high STA) but are less effective than targeted attacks. Since the trigger is not closely associated with each label, the untargeted attack cannot achieve the best attack performance, resulting in a lower

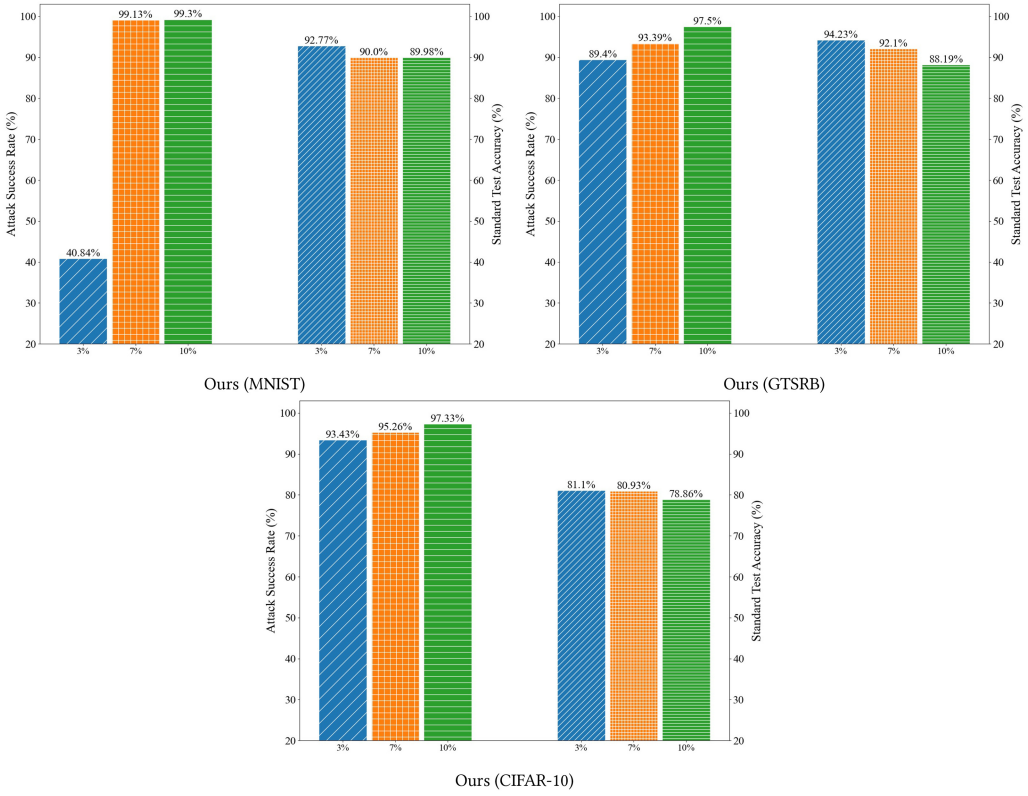


Fig. 7. Performance comparison of different sizes of model-dependent triggers.

Table 3. The Impact of Transparency Value on B³

Transparency value	MNIST		GTSRB		CIFAR-10	
	ASR	STA	ASR	STA	ASR	STA
0	99.13%	90.00%	93.39%	92.10%	95.26%	80.93%
0.1	99.19%	90.50%	93.31%	92.34%	95.26%	80.93%
0.2	98.96%	92.30%	93.32%	92.51%	95.00%	80.97%
0.3	98.95%	92.62%	93.12%	91.64%	95.00%	80.01%
0.4	97.88%	93.71%	92.43%	92.23%	94.88%	80.88%
0.5	97.62%	93.72%	92.26%	90.70%	94.98%	80.62%
0.6	95.65%	95.69%	92.11%	92.29%	92.54%	80.64%
0.7	90.45%	95.52%	91.23%	90.94%	92.13%	81.47%
0.8	90.59%	95.46%	88.59%	91.46%	91.72%	81.03%
0.9	88.76%	96.35%	84.65%	90.20%	90.00%	81.14%

attack success rate than the targeted attack. How to further improve the ASR of untargeted attacks is our future work.

5 ROBUSTNESS TO DEFENSES

We analyze the robustness of B³ against two state-of-the-art defenses, i.e., Pruning [32] and **NeuralCleanse** (NC) [48]. We have reproduced these defense strategies against B³ and demonstrated that B³ is able to resist these defense strategies.

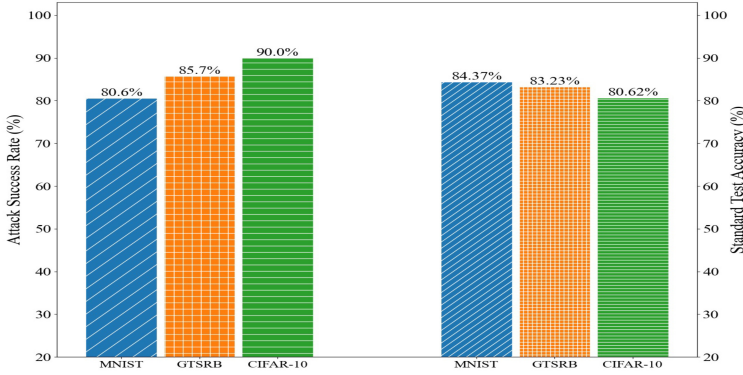


Fig. 8. Performance of untargeted attacks.

Robustness to Pruning. Pruning [32] removes potential backdoors by pruning unnecessary connections of the DNN model. It is based on the assumption that backdoored neurons (i.e., activated by backdoored images) are less active for benign inputs. The pruning defense method does not examine the existence of backdoors and prunes benign models as well. The defender first records each neuron's activation value when feeding benign samples. Then, neurons are removed in ascending order of activation values until the prediction accuracy of the test dataset hits a lower bound.

We apply model pruning to the backdoored models of B^3 on MNIST, GTSRB, and CIFAR-10, as shown in Figure 9. As more neurons are pruned, the prediction accuracy has an obvious decrease, but the ASR of B^3 is still high. It is due to we choose the neuron that has a high activation value when feeding a large number of benign samples (of the targeted label). This shows that B^3 is robust to network pruning.

Robustness to NC. The observation of NC is that the backdoored model requires much smaller perturbations on the input samples to make them misclassified to the targeted category than into other categories. A major advantage of NC is that it requires knowing nothing about the training data. NC inverses a potential trigger for each class to identify if any backdoor trigger is obviously smaller than other generated triggers through outlier detection. The significantly smaller one is considered to be the real trigger. The corresponding label is the targeted label.

When applied to the backdoored models of B^3 , for the MAD anomaly detection, the threshold is 1.4, that is, any label with a MAD larger than 1.4 is the target misclassification label. The results of the MAD of target labels for MNIST, GTSRB, and CIFAR-10 are 0.09, 0.51, and 0.12, which are much lower than 1.4. Therefore, our proposed attacks can evade NC defense. In comparison, we run NC on the example backdoored models given in Reference [48]. It turns out that NC works well and can precisely locate the targeted label.

Robustness to NAD. Neural Attention Distillation (NAD) [30] aims to erase the backdoor from the model. In NAD, the defender first fine-tunes the backdoored model on a small set of benign samples and uses the fine-tuned model as a teacher model. Then, NAD uses the teacher model to distill the backdoored model (student model) through an attention distillation. In this way, the neurons of the backdoor will be aligned with benign neurons associated with meaningful representations. When applying NAD on B^3 , as shown in Figure 10, the ASR of B^3 slightly decreases, which means that B^3 is resistant to NAD defense.

Robustness to CutMix. CutMix [1] cuts off the link between the trigger and the backdoor by image augmentation during training time. It takes pairs of samples and randomly cuts out a small

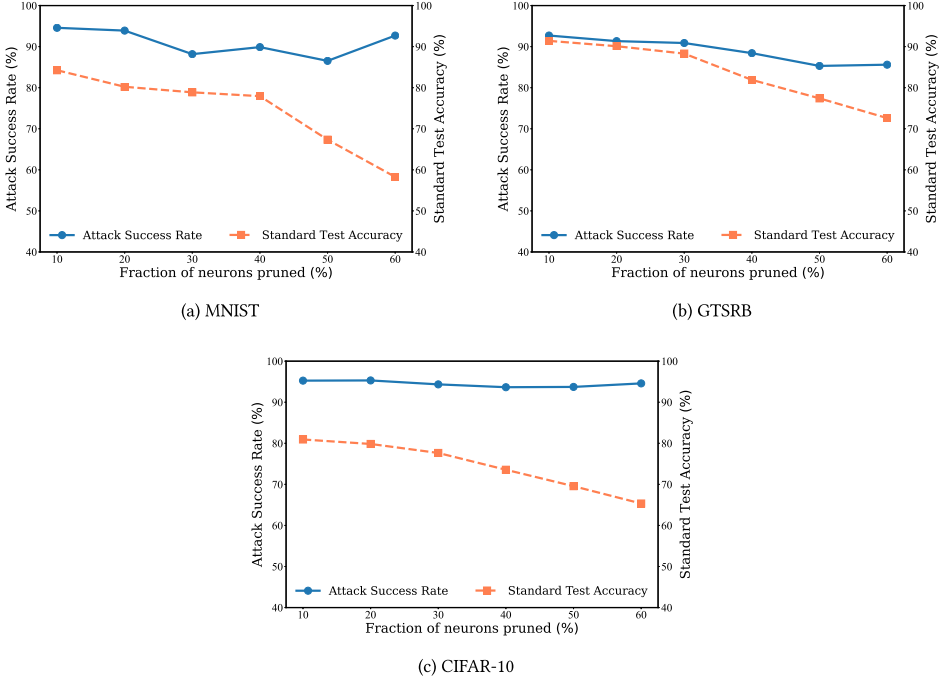
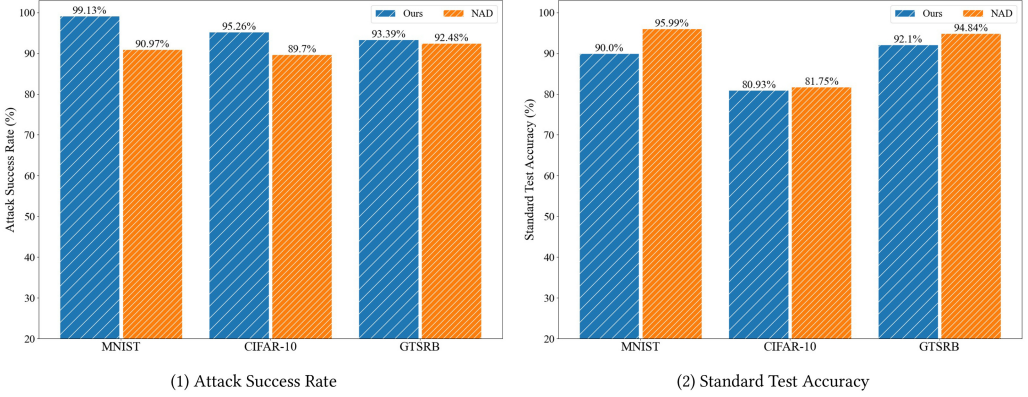


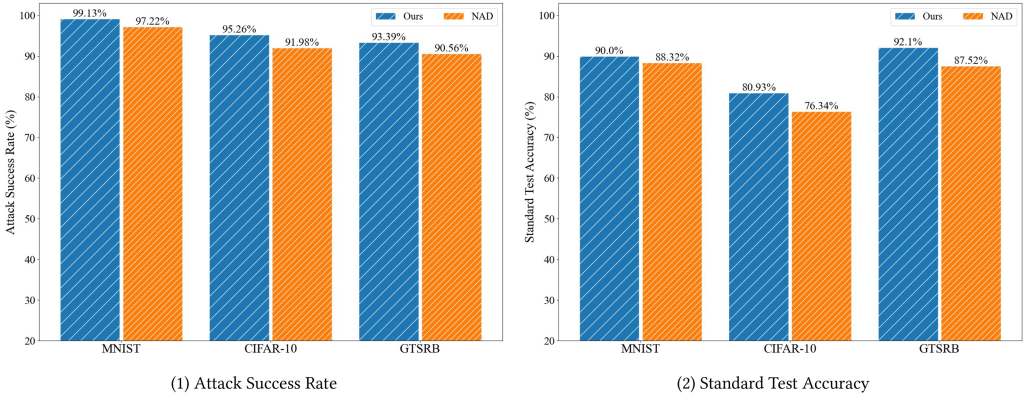
Fig. 9. Results of network pruning.

Fig. 10. Apply NAD to B³.

part of one image and overlays the same part of the other image. At the same time, labels are also fused according to the proportion of the coverage part. CutMix enables the model to make correct classifications from local features and partial views, improving the robustness and generalization of the model. We apply CutMix on B³, and the results are shown in Figure 11. It is shown that CutMix fails to defend B³ and may lead to a significant reduction in STA.

6 RELATED WORK

Backdoor attacks. Backdoor attack aims to inject backdoors into machine learning models so inputs with the trigger will be misclassified to a targeted label or any false labels. To the best

Fig. 11. Apply CutMix to B^3 .

of our knowledge, current backdoor attacks mostly deal with the white-box or grey-box settings, where all or partial information of the victim model is known. Gu et al. [18] proposed BadNets under the white-box setting in which the adversary can control the entire training process, including modifying the training dataset, the configuration of the learning algorithms, and parameters of the model. BadNets used a random trigger to conduct the attacks. Rather than using a static trigger with a fixed location and shape, Salem et al. [43] proposed a dynamic trigger generation algorithm based on generative networks. Li et al. discovered that varying the trigger location or appearance significantly degrades the attack effectiveness [31] and proposed to use random spatial transform on poisoned samples before backdoor injection to enhance attack robustness. Gong et al. [13] proposed to use multi-location patching strategy and multi-trigger backdoor attacks to improve attack robustness in the white-box setting. To improve trigger stealthiness, invisible trigger generation algorithms were proposed in the white-box scenarios [28, 42]. In the grey-box settings, Liu et al. assumed that the attacker has access to the victim model but not the training data [34]. A model-dependent trigger is generated based on the chosen neuron that receives the highest impact from the preceding layer, but the association between the selected neuron and the targeted misclassification class is not considered. Ji et al. [21, 22] assumed that the attacker could control only partial information of the target model, e.g., the classifier architecture or the feature extractor. Unlike other attacks, the trigger used in Reference [21] is a specific sample (e.g., a portrait of the attacker). The appearance of the specific sample will lead to misclassification, e.g., an authorized client as the targeted label. Yao et al. [50] put forward a backdoor attack against transfer learning, in which the attacker embeds latent backdoors to the teacher model, thus the student model inherits the backdoor. Chen et al. [3] demonstrated a backdoor attack that needs neither the training samples nor the victim model, but the attacker needs to be able to poison the training dataset as an insider. Different from these existing works, we deal with the black-box scenario, where the adversary does not know either the model or the training dataset, and we put forward a trigger generation strategy that strengthens the link between the trigger and the backdoor.

There only exist few works [27, 29] that target black-box scenario. Inspired by the DNN interpretation that deep neural network explores edge and color knowledge, Li et al. [27] proposed a black-box backdoor attack against face recognition systems. The attacker uses a color stripe pattern as backdoor trigger that can be generated by modulating LED in a specialized waveform. To inject the backdoor trigger without accessing the dataset, the attacker is assumed to deploy the modified LEDs in advance, so captured backdoored samples can be directly fed into the face-recognition system at the time of the face-registration process. More recently, Li et al. [29]

proposed DeepPayload, which focuses on mobile deep learning apps. DeepPayload can be assembled as a malicious payload and directly embedded into the target model via bytecode rewriting.

Backdoor defenses. Existing defense strategies against backdoor attacks are developed along two key directions, i.e., *online detection* as well as *offline detection* [4]. Online detection aims to identify whether a given input contains the trigger during model running, e.g., by checking runtime irregularities. Offline detection examines whether the model contains backdoors before deployment.

After the DNN has been deployed, it is possible to check whether the incoming inputs contain the trigger. SentiNet [6] considers that an input image has a backdoor trigger if certain contiguous regions have a strong influence on the classification results. Strip [11], another online backdoor detection scheme, has a lower computational cost than SentiNet [6]. Strip replicates the input image multiple times and perturbs each replica by adding a different image. These perturbed replicas are fed into the deep neural network for classification. By measuring the entropy of the predicted classes, the input is determined to be backdoored or not. NIC [35] is another online detection method that leverages the intermediate results and activation values of each layer in DNN to screen inputs with the trigger.

Before deploying the model, it is possible to directly check whether the model has been backdoored. **Activation Clustering (AC)** [2] is based on the intuition that the activations of the DNN should be different for benign inputs and backdoored inputs under the same label. **Neural Cleanse (NC)** [48] is more realistic than AC [2], as it does not require access to the training data. NC is based on the observation that for a backdoored DNN model, it requires much smaller modifications on the inputs to have them misclassified to the target label than into other labels. More recently, **Artificial Brain Stimulation (ABS)** [33] uses **Electrical Brain Stimulation (EBS)** to scan the target model to determine whether it is backdoored or not.

7 CONCLUSIONS

In this article, we design, implement, and evaluate a practical and effective backdoor attack on black-box machine learning models. We demonstrate the feasibility of such attacks by addressing various technical challenges, including the lack of information of the original black-box victim model and the original training dataset. To establish a well-performed substitute model, we propose to construct a cost-effective query dataset based on active learning and adversarial examples. To compensate for the lack of the original training dataset, we leverage model inversion to generate a surrogate training dataset that embodies the most representative features of the original dataset. Furthermore, we propose a novel trigger generation approach that tightens the association between the trigger and the targeted misclassification label. Experiments on both simulated models and the commercial API of Alibaba Cloud Compute service verify the effectiveness of the attack. Such attacks reveal potential security threats to the emerging MLaaS service paradigm and extend the research area of backdoor attacks.

There still exist potential spaces worthy of exploring in the future. It is possible to generalize B³ beyond the vision domain, as our proposed data selection method of model extraction attack can also be applied to other domains. The main challenge may be how to inverse training data in different domains. To the best of our knowledge, existing model inversion attacks are mainly designed for vision domain. How to reconstruct training data in other fields is still an open research direction.

REFERENCES

- [1] Eitan Borgnia, Valeriia Cherepanova, Liam Fowl, Amin Ghiasi, Jonas Geiping, Micah Goldblum, Tom Goldstein, and Arjun Gupta. 2021. Strong data augmentation sanitizes poisoning and backdoor attacks without an accuracy tradeoff. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 3855–3859.

- [2] Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian Molloy, and Biplav Srivastava. 2018. Detecting backdoor attacks on deep neural networks by activation clustering. *arXiv preprint arXiv:1811.03728* (2018).
- [3] Xinyun Chen, Chang Liu, Bo Li, Kimberly Lu, and Dawn Song. 2017. Targeted backdoor attacks on deep learning systems using data poisoning. *arXiv preprint arXiv:1712.05526* (2017).
- [4] Yanjiao Chen, Xueluan Gong, Qian Wang, Xing Di, and Huayang Huang. 2020. Backdoor attacks and defenses for deep neural networks in outsourced cloud environments. *IEEE Netw.* 34, 5 (2020), 141–147.
- [5] Yanjiao Chen, Zhicong Zheng, and Xueluan Gong. 2022. MARNet: Backdoor attacks against cooperative multi-agent reinforcement learning. *IEEE Trans. Depend. Secure Comput.* (2022).
- [6] Edward Chou, Florian Tramèr, Giancarlo Pellegrino, and Dan Boneh. 2018. SentiNet: Detecting physical attacks against deep learning systems. *arXiv preprint arXiv:1812.00292* (2018).
- [7] Jacson Rodrigues Correia-Silva, Rodrigo F. Berriel, Claudine Badue, Alberto F. de Souza, and Thiago Oliveira-Santos. 2018. Copycat CNN: Stealing knowledge by persuading confession with random non-labeled data. In *International Joint Conference on Neural Networks*. IEEE, 1–8.
- [8] L. Deng. 2012. The MNIST database of handwritten digit images for machine learning Research [Best of the Web]. *IEEE Signal Process. Mag.* 29, 6 (2012), 141–142.
- [9] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. 2015. Model inversion attacks that exploit confidence information and basic countermeasures. In *ACM SIGSAC Conference on Computer and Communications Security*. 1322–1333.
- [10] Matthew Fredrikson, Eric Lantz, Somesh Jha, Simon Lin, David Page, and Thomas Ristenpart. 2014. Privacy in pharmacogenetics: An end-to-end case study of personalized warfarin dosing. In *USENIX Security Symposium*. USENIX Association, 17–32.
- [11] Yansong Gao, Chang Xu, Derui Wang, Shiping Chen, Damith C. Ranasinghe, and Surya Nepal. 2019. STRIP: A defence against trojan attacks on deep neural networks. In *IEEE Annual Computer Security Applications Conference*. 113–125.
- [12] Xueluan Gong, Yanjiao Chen, Huayang Huang, Yuqing Liao, Shuai Wang, and Qian Wang. 2022. Coordinated backdoor attacks against federated learning with model-dependent triggers. *IEEE Netw.* 36, 1 (2022), 84–90.
- [13] Xueluan Gong, Yanjiao Chen, Qian Wang, Huayang Huang, Lingshuo Meng, Chao Shen, and Qian Zhang. 2021. Defense-resistant backdoor attacks against deep neural networks in outsourced cloud environment. *IEEE J. Select. Areas Commun.* 39, 8 (2021), 2617–2631.
- [14] Xueluan Gong, Yanjiao Chen, Qian Wang, and Weihang Kong. 2022. Backdoor attacks and defenses in federated learning: State-of-the-art, taxonomy, and future directions. *IEEE Wirel. Commun.* 30, 2 (2022), 114–121.
- [15] Xueluan Gong, Yanjiao Chen, Wang Yang, Qian Wang, Yuzhe Gu, Huayang Huang, and Chao Shen. 2023. REDEEM MYSELF: Purifying backdoors in deep learning models using self attention distillation. In *IEEE Symposium on Security and Privacy (SP'23)*. IEEE Computer Society, 755–772.
- [16] Xueluan Gong, Ziyao Wang, Yanjiao Chen, Meng Xue, Qian Wang, and Chao Shen. 2023. Kaleidoscope: Physical backdoor attacks against deep neural networks with RGB filters. *IEEE Trans. Depend. Secure Comput.* (2023).
- [17] Lior Graham and Yitzhak Yitzhaky. 2019. Blind restoration of space-variant Gaussian-like blurred images using regional PSFs. *Signal, Image Vid. Process.* 13, 4 (2019), 711–717.
- [18] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. BadNets: Evaluating backdooring attacks on deep neural networks. *IEEE Access* 7 (2019), 47230–47244.
- [19] Weik Martin H. 2001. *Additive White Gaussian Noise*. Springer US.
- [20] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. 2017. Densely connected convolutional networks. In *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE Computer Society, 2261–2269.
- [21] Yujie Ji, Xinyang Zhang, Shouling Ji, Xiapu Luo, and Ting Wang. 2018. Model-reuse attacks on deep learning systems. In *ACM SIGSAC Conference on Computer and Communications Security*. 349–363.
- [22] Yujie Ji, Xinyang Zhang, and Ting Wang. 2017. Backdoor attacks against learning systems. In *IEEE Conference on Communications and Network Security*. 1–9.
- [23] Mika Juuti, Sebastian Szlyler, Samuel Marchal, and N. Asokan. 2019. PRADA: Protecting against DNN model stealing attacks. In *IEEE European Symposium on Security and Privacy*. 512–527.
- [24] Manish Kesarwani, Bhaskar Mukhoty, Vijay Arya, and Sameep Mehta. 2018. Model extraction warning in MLaaS paradigm. In *34th Annual Computer Security Applications Conference*. ACM, 371–380.
- [25] Alex Krizhevsky, Geoffrey Hinton, and others. 2009. Learning multiple layers of features from tiny images. (2009).
- [26] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [27] Haoliang Li, Yufei Wang, Xiaofei Xie, Yang Liu, Shiqi Wang, Renjie Wan, Lap-Pui Chau, and Alex C. Kot. 2020. Light can hack your face! Black-box backdoor attack on face recognition systems. *arXiv preprint arXiv:2009.06996* (2020).

- [28] Shaofeng Li, Minhui Xue, Benjamin Zi Hao Zhao, Haojin Zhu, and Xinpeng Zhang. 2019. Invisible backdoor attacks on deep neural networks via steganography and regularization. *arXiv preprint arXiv:1909.02742* (2019).
- [29] Yuanchun Li, Jiayi Hua, Haoyu Wang, Chunyang Chen, and Yunxin Liu. 2021. DeepPayload: Black-box backdoor attack on deep learning models through neural payload injection. In *IEEE/ACM 43rd International Conference on Software Engineering*. IEEE, 263–274.
- [30] Yige Li, Nodens Koren, Lingjuan Lyu, Xixiang Lyu, Bo Li, and Xingjun Ma. 2021. Neural attention distillation: Erasing backdoor triggers from deep neural networks. In *International Conference on Learning Representations*. OpenReview.net.
- [31] Yiming Li, Tongqing Zhai, Baoyuan Wu, Yong Jiang, Zhifeng Li, and Shutao Xia. 2020. Rethinking the trigger of backdoor attack. *arXiv preprint arXiv:2004.04692* (2020).
- [32] Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2018. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *International Symposium on Research in Attacks, Intrusions, and Defenses*. Springer, 273–294.
- [33] Yingqi Liu, Wen-Chuan Lee, Guanhong Tao, Shiqing Ma, Yousra Aafer, and Xiangyu Zhang. 2019. ABS: Scanning neural networks for backdoors by artificial brain stimulation. In *ACM SIGSAC Conference on Computer and Communications Security*. 1265–1282.
- [34] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. 2018. Trojaning attack on neural networks. In *Annual Network and Distributed System Security Symposium*. The Internet Society.
- [35] Shiqing Ma, Yingqi Liu, Guanhong Tao, Wen-Chuan Lee, and Xiangyu Zhang. 2019. NIC: Detecting adversarial samples with neural network invariant checking. In *Annual Network and Distributed System Security Symposium*. The Internet Society.
- [36] Ari S. Morcos, David G. T. Barrett, Neil C. Rabinowitz, and Matthew Botvinick. 2018. On the importance of single directions for generalization. In *6th International Conference on Learning Representations*. OpenReview.net.
- [37] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. 2019. Knockoff nets: Stealing functionality of black-box models. In *IEEE Conference on Computer Vision and Pattern Recognition*. 4954–4963.
- [38] Soham Pal, Yash Gupta, Aditya Shukla, Aditya Kanade, Shirish Shevade, and Vinod Ganapathy. 2020. ACTIVETHIEF: Model extraction using active learning and unannotated public data. In *AAAI Conference on Artificial Intelligence*. AAAI Press, 865–872.
- [39] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. 2017. Practical black-box attacks against machine learning. In *ACM Asia Conference on Computer and Communications Security*. 506–519.
- [40] Erwin Quiring, Daniel Arp, and Konrad Rieck. 2018. Forgotten siblings: Unifying attacks on machine learning and digital watermarking. In *IEEE European Symposium on Security and Privacy*. 488–502.
- [41] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael S. Bernstein, Alexander C. Berg, and Fei-Fei Li. 2015. ImageNet large scale visual recognition challenge. *Int. J. Comput. Vis.* 115, 3 (2015), 211–252.
- [42] Aniruddha Saha, Akshayvarun Subramanya, and Hamed Pirsiavash. 2020. Hidden trigger backdoor attacks. In *AAAI Conference on Artificial Intelligence*. AAAI Press, 11957–11965.
- [43] Ahmed Salem, Rui Wen, Michael Backes, Shiqing Ma, and Yang Zhang. 2020. Dynamic backdoor attacks against machine learning models. *arXiv preprint arXiv:2003.03675* (2020).
- [44] Ozan Sener and Silvio Savarese. 2018. Active learning for convolutional neural networks: A core-set approach. In *6th International Conference on Learning Representations*. OpenReview.net.
- [45] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. 2012. Man vs. Computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Netw.* 32 (2012), 323–332.
- [46] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2014. Intriguing properties of neural networks. In *2nd International Conference on Learning Representations*. OpenReview.net.
- [47] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2016. Stealing machine learning models via prediction APIs. In *USENIX Security Symposium*. USENIX Association, 601–618.
- [48] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. 2019. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *IEEE Symposium on Security and Privacy*. 707–723.
- [49] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv:cs.LG/1708.07747*.
- [50] Yuanshun Yao, Huiying Li, Haitao Zheng, and Ben Y. Zhao. 2019. Latent backdoor attacks on deep neural networks. In *ACM SIGSAC Conference on Computer and Communications Security*. 2041–2055.

- [51] Honggang Yu, Kaichen Yang, Teng Zhang, Yun-Yun Tsai, Tsung-Yi Ho, and Yier Jin. 2020. CloudLeak: Large-scale deep learning models stealing through adversarial examples. In *Network and Distributed Systems Security Symposium*. The Internet Society.
- [52] Sergey Zagoruyko and Nikos Komodakis. 2016. Wide residual networks. In *British Machine Vision Conference*. BMVA Press.

Received 20 December 2021; revised 17 October 2022; accepted 15 June 2023